

Advanced Concepts in python

- 1) Decorators
- 2) generators
- 3) iterators

Pandas

- 1) Data Analyst Interview (Python, SQL, Pandas)
- 2) PySpark →
- 3) Data scientist →

What is Pandas?

Pandas is an **open-source Python library** used for data manipulation and analysis. It provides **data structures** like **Series** (1D) and **DataFrame** (2D) to handle structured data efficiently.

Where is Pandas Used?

Pandas is widely used in **data science, analytics, and machine learning** for:

- ☒ **Data Cleaning & Preprocessing** – Handling missing values, duplicates, and formatting data.
- ☒ **Data Analysis** – Performing statistical calculations and data summarization.
- ☒ **Data Visualization** – Plotting data using built-in functions and integration with Matplotlib & Seaborn.
- ☒ **Data Transformation** – Merging, reshaping, and filtering large datasets.
- ☒ **Time Series Analysis** – Handling time-indexed data like stock prices or IoT data.
- ☒ **Big Data Handling** – Works well with CSV, Excel, SQL databases, and JSON files.

Data Manipulation

Empid	Emp Name	Emp Sal
123	John Smith	1000
345	Kanna Jilla	2000
987	Ravi Kohli	3000

⇒

Empid	Emp Address	Emp Last	Emp Sal
123	Paris	Smith	1000
345	100	Jilla	2000
987	100	Kohli	3000

Structured :-

- 1) Excel ✓
- 2) CSV ✓
- 3) Table in DB (SQL)
- 4) PostgreSQL ✓

Unstructured :-

- 1) MongoDB
- 2) Cassandra
- 3) AWS S3
- 4) Video ✓
- 5) Images ✓

{ - - - }

```
l=[[1,2,3,4],[5,6,7,8],[2,3,4,5]]
```

```
df = pd.DataFrame(l)  
df
```

A hand-drawn diagram of a DataFrame table. The table has 4 columns and 3 rows. The columns are labeled 0, 1, 2, 3. The rows are labeled 0, 1, 2. The data is as follows:

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	2	3	4	5

Annotations: A box is drawn around the first two columns (0 and 1). An arrow points from this box to the text "Series (10)". Another box is drawn around the first two rows (0 and 1). An arrow points from this box to the text "DataFrame (20)".

Series (10) [1, 5, 2] - 10

DataFrame (20)
[(1, 2, 3, 4),
 (5, 6, 7, 8),
 (2, 3, 4, 5)]

Few Important Methods

19 February 2025 07:28

.loc & .iloc

.apply()

----> Modify the Runtime column

----> Modify the Gross column

.rename()

.De-nesting Genre for top 10 movies based on Meta_score

.duplicated()

.isnull()

.notnull()

.fillna()

.dropna()

.drop_duplicates()

.drop()

.loc & .iloc

.

→ Both are used to filter / select / ^{update} rows / columns in a dataframe.

diff .loc & .iloc

→ .loc uses labels

→ .iloc uses indexes / positions

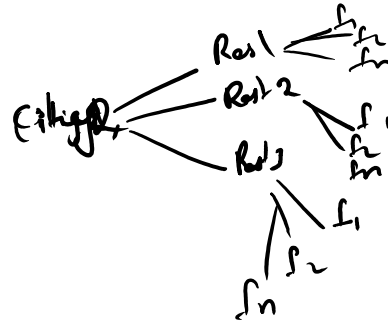
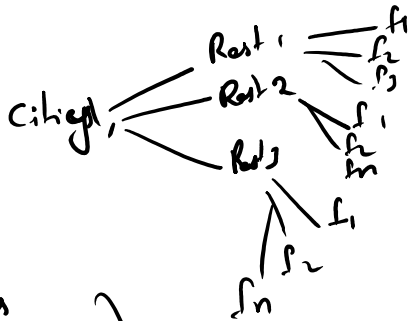
Syntax

df.loc[
 ↓
rows,
 ↓
columns]

df.iloc[
 ↓
rows,
 ↓
columns]

→ helps in dividing dataset into groups, on top of it you can perform transformations/operations on each group.

Ex: -



Categories
Biryani
Burger
Snacks
breakfast

Sample data

Find cat of food item with highest sales!

Sample dataset -

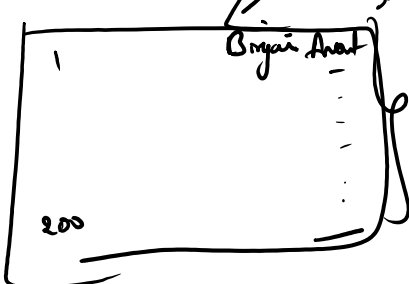
Swiggy table

categories

S.No	Food item	Rest Name	Rating	City Name	Category	Amount
1	f1	R1	9.5	Kolkata	Biryani	250
2	f2	R1	8.5	Kolkata	breakfast	90
...						

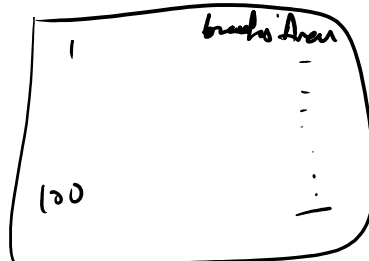
100 rows

1000 x 7
#rows #columns



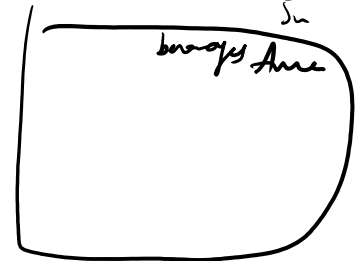
200 x 7
#rows #columns

Sum(Amount)



100 x 7

Sum(Amount)



300 x 7

100 x 7

300 x 7

More

cat values (amount = False)

65

751006

sat values (use = Fabc)

()

סכסוכי 7

(5 75,000
(sum = mcgys $\Rightarrow$ subg) $\Rightarrow$ transformations)

Swt	Ry	R+	Gene Actions	Tidy	one
			Action		
			Achilles		
			Achilles		
			Animation		
			Animals		

2) $\mu_p - \mu_n$

			Wesley	
			Wesley	

```
movies.groupby("Director")["No_of_Votes"].sum()
```

1000x7

10 Directors

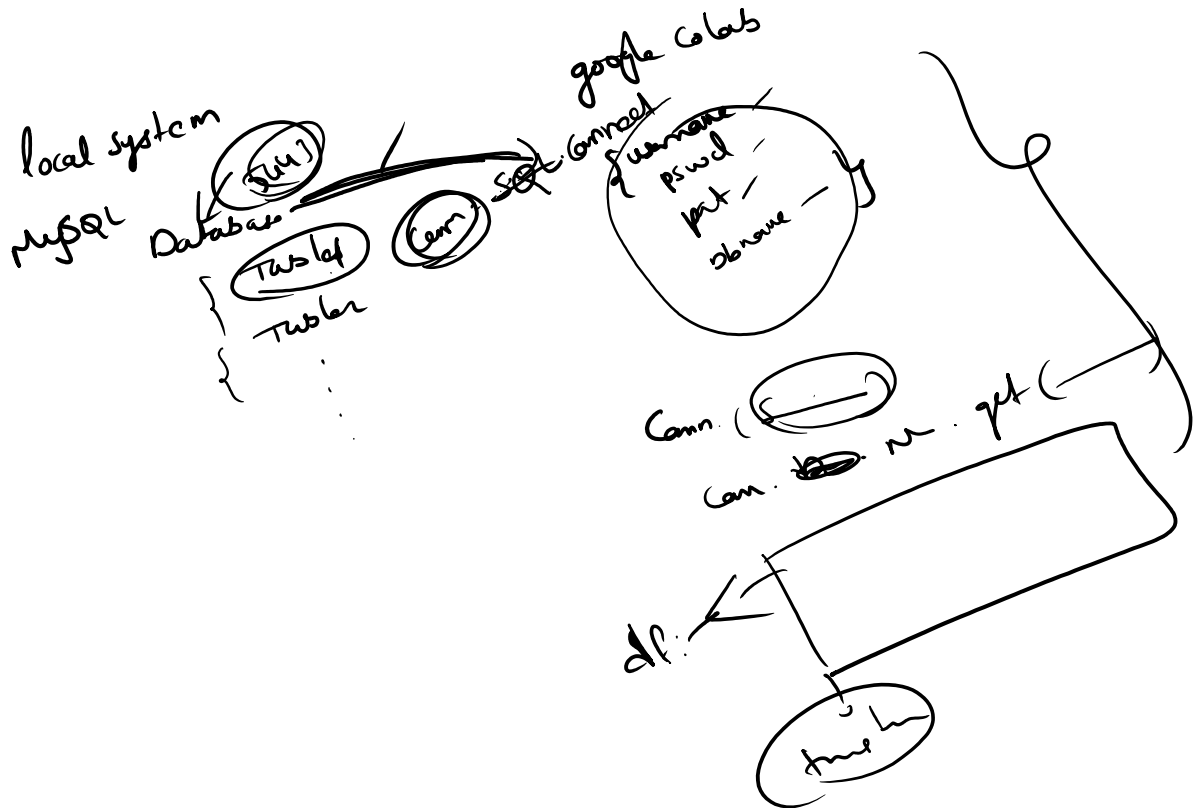
			No. of	Director
1				Don
2				Chris
10				James

Director	No. of	Direct sum
		= 10
		Don
		Don

10x1

dir	Movie sum
01	—
02	—
010	—

dim	Num dim
01	—
02	—
...	...
010	—



====agg
 max
 min
 sum
 mean
 mode
 std
 var
 count
 size - group wise count

first - first row in every group

last - last row in every group

nth(num) - if num = 5, 5th row in every group

get_group("Horror") - returns Horror group related data

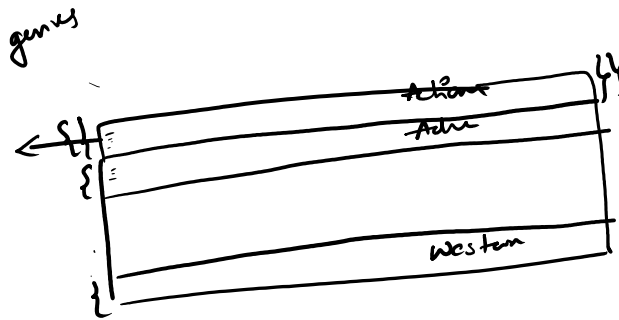
groups - returns dict with keys as group names and values as index positions

describe() - similarly like pandas df on groups it will summarize every column(count, mean, std, 25% etc)

sample(2) - randomly picks 2 rows on each group

nunique() - gives column wise unique count

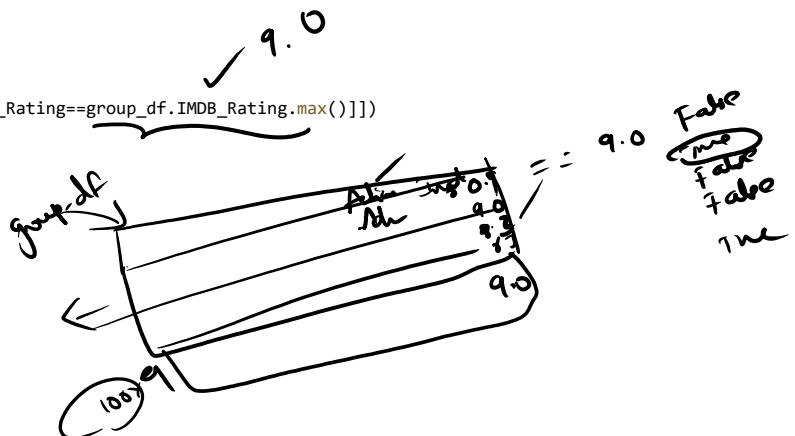
for group, group_df in genres:



```
# looping on groups
#find highest rated movie from each genre
df=pd.DataFrame(columns=movies.columns)
for group, group_df in genres:
    df = pd.concat([df,group_df[group_df.IMDB_Rating==group_df.IMDB_Rating.max()]])
```

Divide & conquer
group_df.IMDB_Rating.max()

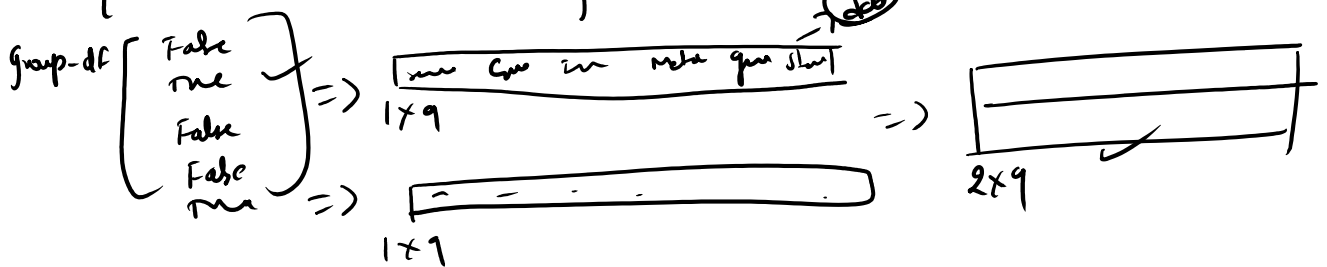
9.0



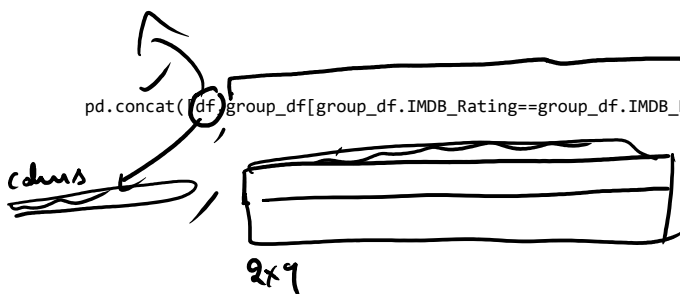
group_df.IMDB_Rating==group_df.IMDB_Rating.max()

False
True
False
False

group_df[group_df.IMDB_Rating==group_df.IMDB_Rating.max()]



pd.concat([df,group_df[group_df.IMDB_Rating==group_df.IMDB_Rating.max()]]) df

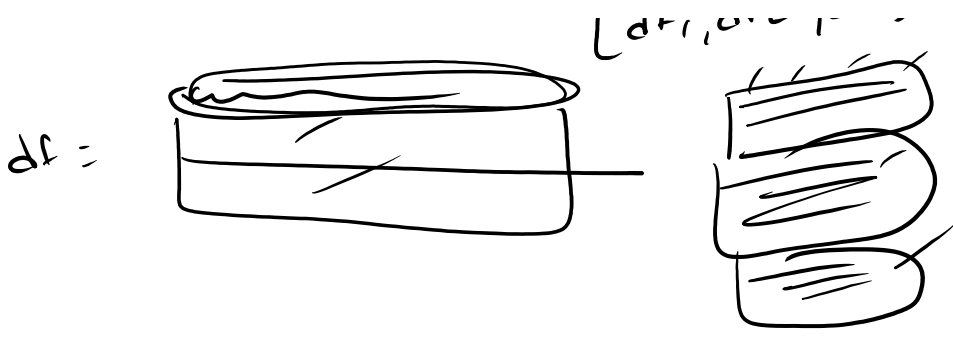


" + "data" => "data"

0+1 => 1

[df, df, df]

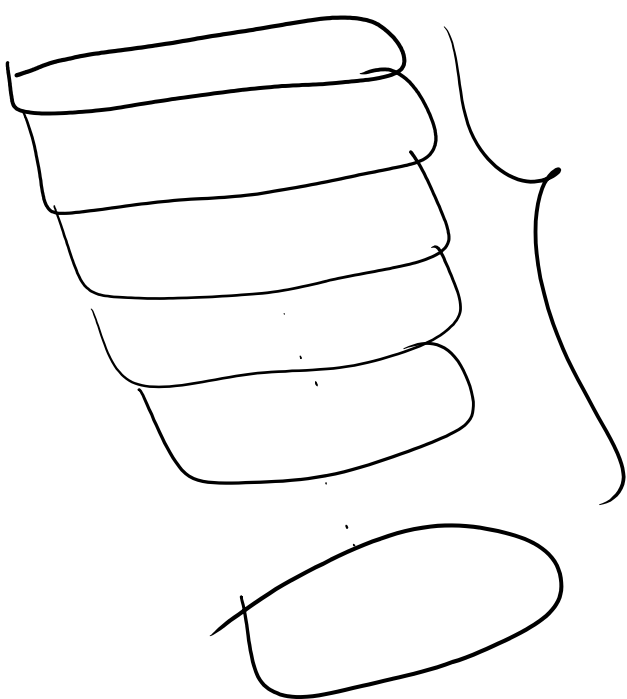
(df, df, df)



```
#using apply method
def genre_highest_rated(group_df):
    return group_df[group_df.IMDB_Rating==group_df.IMDB_Rating.max()]
genres.apply(genre_highest_rated)
```

✓ group data

↑ group-wise



```
# groupby on multiple cols
movies.groupby(["Director", "Star1"])
```

data

Sum AS
Sum AS
Sum AS
4

Director	Star1	Star2
A	D1	S1
B	D1	S1
C	D1	S1

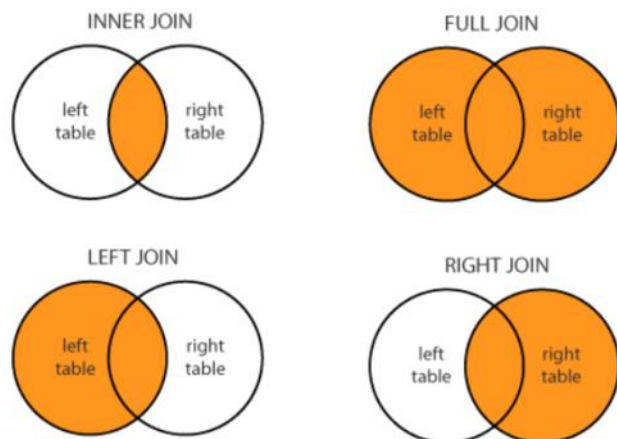
D1	S2
D1	S2
D1	S2

concat, merge, visualization

28 February 2025 07:24

Concat - It allows us to combine 2 or more data frames along horizontal and vertical ✓

Merge - It is also another way to combine data frames ✓



Difference Between concat() and merge() in Pandas

Feature	concat()	merge()
Purpose	Combines DataFrames along rows (axis=0) or columns (axis=1)	Joins DataFrames based on key columns (like SQL JOIN)
How it Works	Simply stacks DataFrames (row-wise or column-wise)	Matches data based on common columns or indexes
Default Behavior	Does not match values, just appends	Matches rows based on key columns
Best Use Case	When combining similar DataFrames (stacking datasets)	When you need relational joins (similar to SQL joins)

Data Visualization(Using Matplotlib, Seaborn):

What is Matplotlib?

Matplotlib is a powerful Python library for creating static, animated, and interactive visualizations. It is often used for plotting line charts, bar charts, scatter plots, and histograms.

What is Seaborn?

Seaborn is built on top of Matplotlib and provides a more visually appealing and statistical way to create plots. It simplifies complex visualizations like heatmaps, box plots, and violin plots.

Concat:

fec ✓

csc ✓

fec

Concat:

fcc ✓

csc ✓

fcc

df1 s-id, s-name, iq

df2 s-id, s-name, iq

df3 s-id, s-name, iq

? input pandas as pd

df = fcc.csc.pd.concat([df1, df2])

s-id, s-name, iq

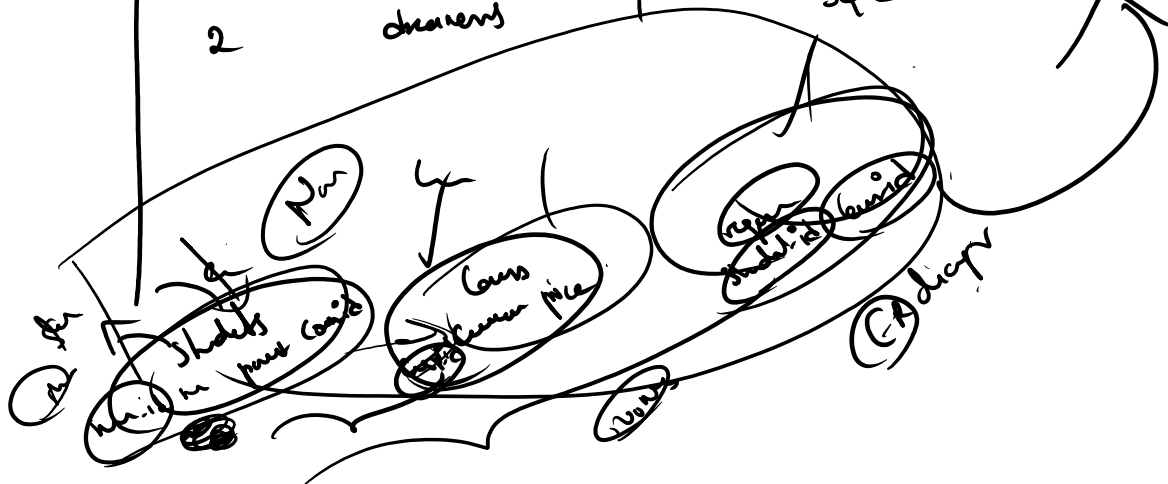
0
10
0
80

Education website Data Modelling :-

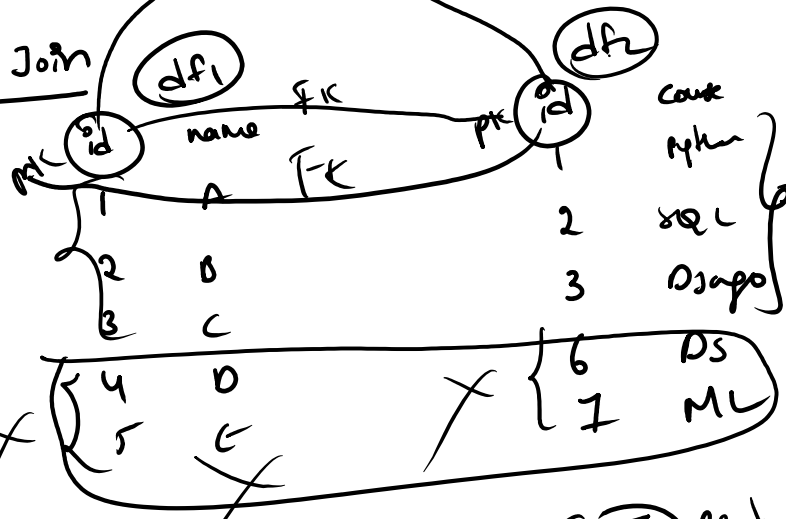
Self → more space
→ Course queries
time management

600

Student-id	Student-name	Marked-id	CourseName	price	Month
1	data	23	python	2000	nov
2	chemistry		SQL	2000	dec



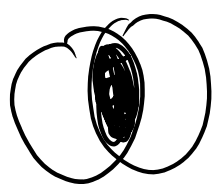
Inner Join



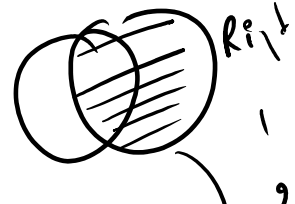
on=id

Inner Join

name	Course
A	python
B	SQL
C	DS Algo



df1 left join df2 on=id



name A

Course python SQL DS Algo

df, df, let join

df,

left join (df)
on id

	A
1	B
2	C
3	D
4	E
5	E

pk
sec
2
Name
Name

2

3

6

7

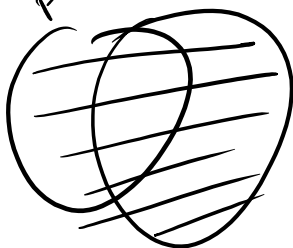
C

Na

Na

sec
Name
DS
ML

Outer
Full



1

2

3

4

5

6

7

A

B

C

D

E

Na

Na

ML

DS

Na

Na

Na

DS

ML

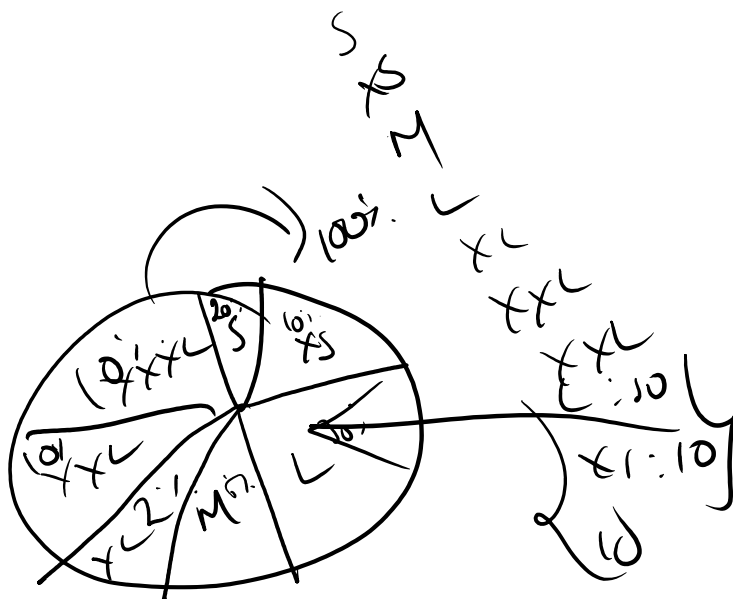
Matplotlib: A Low-Level Plotting Library

- Matplotlib is a **low-level** library that gives **full control** over every aspect of a plot.
- It works like a **canvas**, where you can modify everything, including:
 - Colors
 - Line styles
 - Axes properties
 - Legends
 - Grid styles
 - Annotations
- Since it provides raw access to plot elements, you can create **highly customized visualizations**.

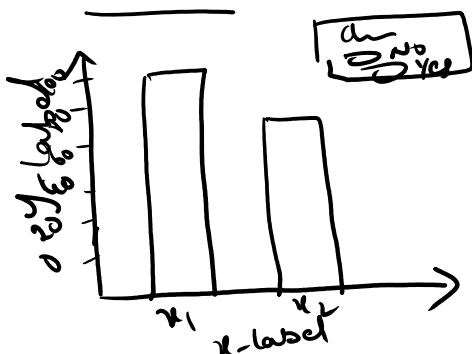
Seaborn: A High-Level Statistical Library

- Seaborn is built **on top of Matplotlib** but is designed to work with **structured datasets (DataFrames)**.
- It **automates** many visualizations, making plots **easier to create and visually appealing by default**.
- However, this automation **limits deep customization** because Seaborn **abstracts away** some of Matplotlib's low-level details.

- 1) Bar plot ✓
- 2) Scatter plot ✓
- 3) Box plot ✓
- 4) histogram ✓
- 5) heat map ✓
- 6) Violin plot ✓
- 7) pie chart ✓

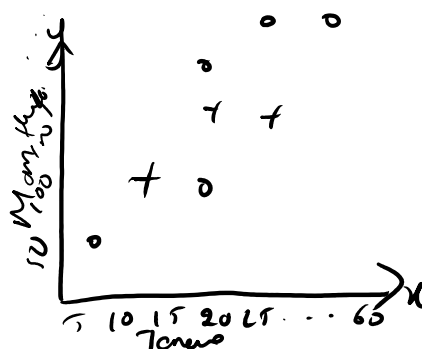


1) Bar plot: —



2) Scatter plot

hue = "class"



3) Box plot

Monthly-charges = [10, 75, 120, 25, 35, 100 ...]

= [25, 35, 50, 75, 100, 120 ...]

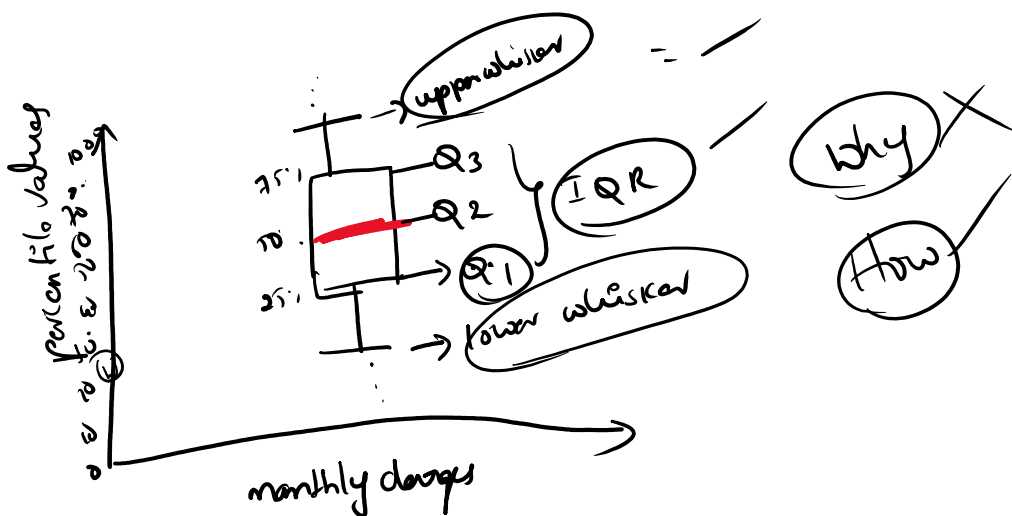
100 ← (25%) (50%) (75%) → 200

IQR (Inter Quartile range) = 75% - 25%

Q₁ = 25%
Q₂ = 50%
Q₃ = 75%

upper-whisker = Q₃ + 1.5 IQR
= 75% + 1.5 * (75% - 25%)

lower-whisker = Q₁ - 1.5 (IQR)
= 25% - 1.5 (75% - 25%)



4) histogram

bins

Count

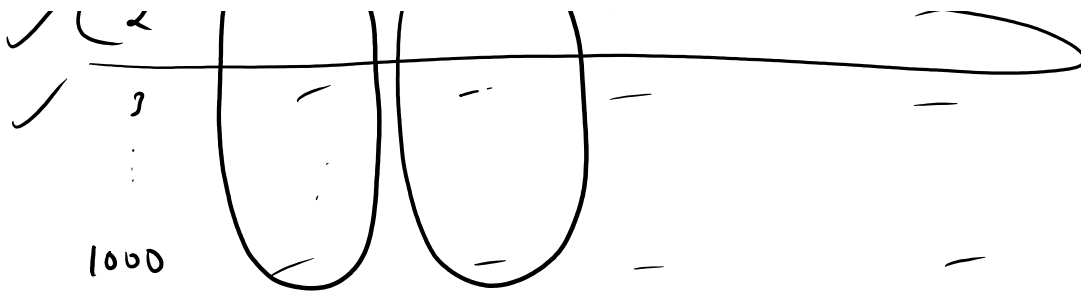
Ex: - 72 hrs (dd-mm-yyyy HH:MM:SS)

Timestamp	Temp (°C)
03-03-2025 23:24:00	16.2
03-03-2025 23:20:45	17.2

The best 500 students

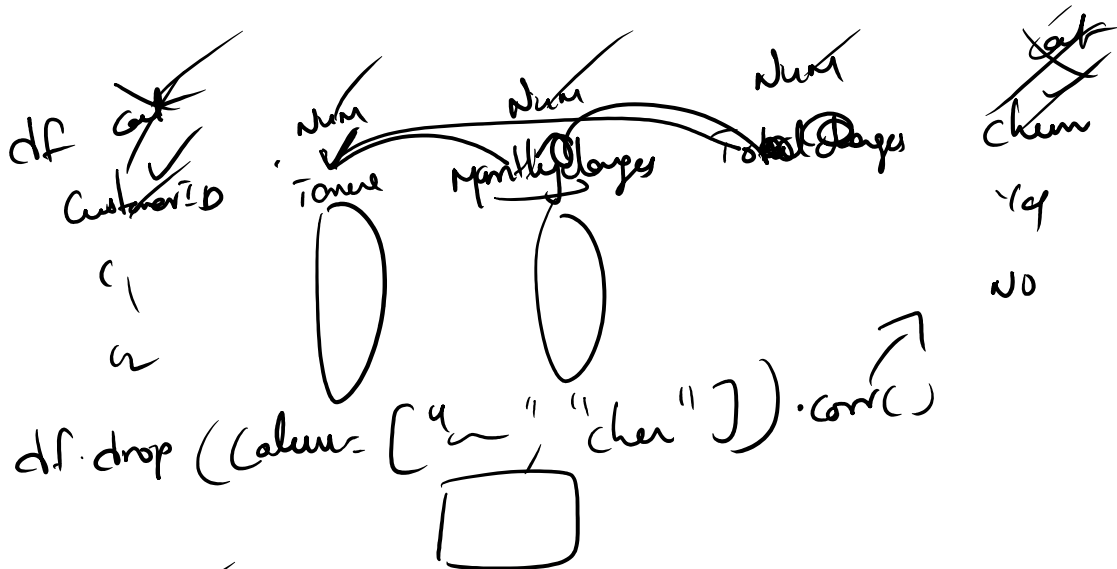
Ex: -2 (categorical)

Sid	dept	T-S
1	ECE	L
2	ECE	M
3	CSC	S



$f \begin{matrix} h \uparrow & w \uparrow \\ h \downarrow & w \downarrow \end{matrix}$
 $h \times (w) = w$

- 1) Pearson correlation coefficient //
- 2) Co-variance //
- 3) Spearman — //



$\text{corr}(\text{Tone}(\text{US}), \text{Math days}) \Rightarrow$
 $\text{corr}(\text{Tone}(\text{US}), \text{Total days}) \Rightarrow$
 $\text{corr}(\text{Tone}(\text{US}), \text{Tone}) =$

$\text{corr}(\text{Math, Total}) \Rightarrow$
 $\text{corr}(\text{Math, Math}) \Rightarrow$
 $\text{corr}(\text{Total, Total}) \Rightarrow$

Tone Math Total

	True	Math	Total
True	✓		
Math		✓	✓
Total	✓	✓	✓

2-5 5-10 10-15

